COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	October 1, 2026	**DATE SUBMITTED:**	October 2, 2026
NAME:	Benj Federic C. De Leon.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Hands-on Activity 10.1 Graph

1. PROCEDURE OUTPUT

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

A.1. Create a Graph

Following are the basic operations that we can perform on the graph data structure:

- Add a vertex: Adds vertex to the graph.
- Add an edge: Adds an edge between the two vertices of a graph.
- Display the graph vertices: Display the vertices of a graph.

Ap1.cpp

```
#include "Ap1.h"

//ILO A
void test_AdjList()
{
    // graph edges array.
    graphEdge edges[] = {
    // (x, y, w) -> edge from x to y with weight w
    {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
    };

    int N = 6; // Number of vertices in the graph

    // calculate number of edges
    int n = sizeof(edges)/sizeof(edges[0]);

    // construct graph
    DiaGraph diagraph(edges, n, N);

    // print adjacency list representation of graph
```

```
    std::cout << "Graph adjacency list " << std::endl
              << "(start_vertex, end_vertex, weight):" << std::endl;

    for (int i = 0; i < N; i++)
    {
    // display adjacent vertices of vertex i
    display_AdjList(diagraph.head[i], i);
    }
}

int main()
{
    using T = unsigned;

    std::cout << "ILO A: Adjacency List:" << std::endl;
    test_AdjList();

    return 0;
}
```

Ap1.h

```
#ifndef Ap1_H
#define Ap1_H

#include <iostream>

// ILO A
// stores adjacency list items
struct adjNode {
int val, cost;
adjNode* next;
};

// structure to store edges
struct graphEdge {
int start_ver, end_ver, weight;
```

```
};

class DiaGraph {
// insert new nodes into adjacency list from given graph
adjNode* getAdjListNode(int value, int weight, adjNode* head) {
adjNode* newNode = new adjNode;
newNode->val = value;
newNode->cost = weight;

newNode->next = head;   // point new node to current head
return newNode;
}

int N; // number of nodes in the graph

public:
adjNode **head; //adjacency list as array of pointers

// Constructor
DiaGraph(graphEdge edges[], int n, int N) {
// allocate new node
head = new adjNode*[N]();
this->N = N;

// initialize head pointer for all vertices
for (int i = 0; i < N; ++i)
head[i] = nullptr;

// construct directed graph by adding edges to it
for (int i = 0; i < n; i++) {
int start_ver = edges[i].start_ver;
int end_ver = edges[i].end_ver;
int weight = edges[i].weight;

// insert in the beginning
adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
```

```
// point head pointer to new node
head[start_ver] = newNode;
}
}

// Destructor
~DiaGraph() {
for (int i = 0; i < N; i++) {

// FIX: head[i] is a linked list of single nodes,
// so delete them one by one.
adjNode* ptr = head[i];
while (ptr != nullptr) {
adjNode* temp = ptr;
ptr = ptr->next;
delete temp;
}
}
delete[] head;
}
};

// print all adjacent vertices of given vertex
inline void display_AdjList(adjNode* ptr, int i)
{
while (ptr != nullptr) {
std::cout << "(" << i << ", " << ptr->val
<< ", " << ptr->cost << ") ";
ptr = ptr->next;
}
std::cout << std::endl;
}

#endif // Ap1_H
```

Output

```
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

Code Analysis:

The program stores a directed, weighted graph as an array of linked lists. Each slot of head[] is one vertex, and each node in its list is an edge (end vertex and weight). New edges are added at the front of the list, so each vertex prints its edges in reverse order. display_AdjList walks each list and prints (start, end, weight).

Error: Wrong delete in the destructor. The original was delete[] head[i];. Each node was created with a single new adjNode, so delete[] is the wrong form, and it only frees the first node and leaks the rest. I changed it to loop through the list and delete each node one by one, then delete[] head at the end.

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

B.1. Depth-First Search

Bp1_DFS.cpp

```cpp
#include "Bp1.h"

int main()
{
using T = unsigned;
test_DFS<T>();
return 0;

    return 0;
}
```

Bp1.h

```cpp
#ifndef Bp1_H
#define Bp1_H

// ILO B
// Step 1:
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

// Step 2:
template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;
    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

// Step 3:
template <typename T>
```

```cpp
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}

// Step 4:
template <typename T>
class Graph
{
public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N)
    {
    }

    // Return number of vertices in the graph
    auto vertices() const
    {
        return V;
    }

    // Return all edges in the graph
    auto &edges() const
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
```

```
        // Check if the source and destination vertices are within
range
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    // Overloads the << operator so a graph be written directly to a
stream
    // Can be used as std::cout << obj << std::endl
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U>
&G);

private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
};

//
============================================================================
```

```
// B.1 Depth-First Search
//
================================================================================
// Step 5:
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1); // Assume that DFS always starts from vertex ID 1
    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            {
                // If the vertex hasn't been visited, insert it in the
stack.
                if (visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}

//Step 6:
// DFS reference graph (edge weights are null/0)
namespace dfs_graph {
```

```cpp
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}
}

// Step 7:
template <typename T>
void test_DFS()
{
    // Create an instance of and print the graph
    auto G = dfs_graph::create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}
```

```
#endif // Bp1_H
```

Output

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010-C
) { .\Bp1_DFS }
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

Code Analysis

The program stores the graph as an edge list and uses a stack to go as deep as possible before backing up. A set keeps track of visited vertices so none is visited twice. It starts at vertex 1 and produces the order 1, 5, 8, 6, 7, 3, 4, 2. Neighbors are pushed in order, so the last one pushed is visited first.

Error: Its operator<< already has return os; and uses U in the friend declaration. These two lines are what the BFS code was missing.

B.2. Breadth-First Search

Bp2_BFS.cpp

```cpp
#include "Bp2.h"

int main()
{
using T = unsigned;
test_BFS<T>();
return 0;

return 0;
}
```

Bp2.h

```cpp
#ifndef Bp2_H
#define Bp2_H

// B.2 Breadth-First Search
// Step 1:
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>
```

```cpp
template <typename T>
class Graph;

// Step 2:
template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

// Step 3:
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;   // FIX: this line was missing
}
```

```cpp
// Step 4:
template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const
    {
        return V;
    }

    auto &edges() const
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
```

```
    }

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U>
&G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

// Step 6:
template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest)
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1); // Assume that BFS always starts from vertex ID 1
    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}

// Step 5:
// BFS reference graph (with edge weights)
```

```
namespace bfs_graph {
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}
}

// Step 7:
template <typename T>
void test_BFS()
{
    // Create an instance of and print the graph
    auto G = bfs_graph::create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
```

```
        std::cout << v << std::endl;
}

#endif // Bp2_H
```

Output

```
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
PS C:\Users\Benj Federic De Leon\Documents\CPE21S1\CPE 010
```

Code Analysis

The program uses the same edge-list graph as DFS, but it uses a queue instead of a stack. It starts at vertex 1 and visits all neighbors of a vertex before going deeper, so it moves level by level. A set makes sure each vertex is visited only once. The edge weights are stored but not used. The visit order is 1, 2, 5, 4, 8, 3, 6, 7.

Error: Missing return os; in operator<<. The function returned nothing, which is undefined behavior. I added the line. Friend declaration reused T. Inside Graph<T>, template <typename T> friend ... reuses the name and fails to compile. I renamed it to template <typename U>.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

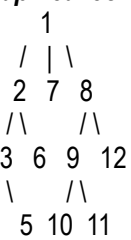
1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.

Depth-First Search (DFS) is the best choice. It follows one path as far as it can, backtracks when it reaches a dead end, and then explores the other unvisited neighbors from the same vertex. This matches the described trip exactly. It uses a stack, or recursion, to remember where to go back to.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

The tree equivalent of DFS is **pre-order traversal**. It visits a node first and then each subtree from left to right. (In-order and post-order are also depth-first, but pre-order is the closest match.)

***Graphical comparison*, using the tree in the activity sheet:**



	Graph DFS	**Tree pre-order**
Start	chosen vertex	root
Goes	deep first, then backtracks	deep first, then backtracks
Memory	stack + visited set	recursion stack
Visited set	needed (graphs have cycles)	not needed (trees have no cycles)
Order here	1 2 3 4 5 6 7 8 9 10 11 12	1 2 3 4 5 6 7 8 9 10 11 12

***Pseudocode**:*

```
preorder(node):

  if node is null: return
  visit(node)
```

for each child c of node:
preorder(c)

Code:

```
struct TreeNode { int val; std::vector<TreeNode*> children; };

void preorder(TreeNode* node) {
    if (node == nullptr) return;
    std::cout << node->val << " ";      // visit first
    for (TreeNode* c : node->children)  // then each subtree
        preorder(c);
}
```

3. In the performed code, what data structure is used to implement the Breadth First Search?

A queue is used. Vertices are taken out in the order they were added, which gives the level-by-level order. A std::set tracks visited vertices, and a std::vector stores the visit order.

4. How many times can a node be visited in the BFS?

Once. A vertex can be added to the queue several times. But the visited set check makes sure it is processed only once.

Conclusion:

In this activity, I learned how to build a graph using an adjacency list and traverse it with DFS (stack) and BFS (queue). In the procedure, I ran all three programs and fixed the errors in them: the wrong delete[] in the destructor, the missing return os;, and the repeated T in the BFS code. The outputs showed that DFS goes deep before backtracking, while BFS goes level by level. In the supplementary activity, I learned that DFS suits exploring and backtracking, that its tree equivalent is pre-order traversal, and that BFS visits each vertex only once. I think I did well overall, but I need to read code more carefully to catch errors and practice more graph problems.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Concepts

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Was used as a guide explaining concepts

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.